

scripts_sql_ant

Scripts sql do projeto

- Scripts sql do projeto
 - Script SQL PostgreSQL para criar banco de dados e tabelas
 - Script SQL PostgreSQL para criar os relacionamentos
 - Script SQL PostgreSQL para criar os índices das tabelas
 - Compartilhar o acesso ao banco de dados
 - Ferramenta que pode ser usada para executar os scripts acima?
 - Referências

Script SQL PostgreSQL para criar banco de dados e tabelas

```
CREATE DATABASE assistente_virtual;

-- Criação das tabelas

CREATE TABLE operadores (
  id INTEGER PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  login VARCHAR(50) NOT NULL,
  password VARCHAR(50) NOT NULL,
  telefone VARCHAR(20),
  UNIQUE (login) -- índice único para login
);

CREATE TABLE hospitais (
  id INTEGER PRIMARY KEY,
  nome VARCHAR(50),
  telefone VARCHAR(20)
);

CREATE TABLE status_da_agenda_ou_consulta (
  id INTEGER PRIMARY KEY,
  nome VARCHAR(20)
);

CREATE TABLE medicos (
  id INTEGER PRIMARY KEY,
  id_operadores INTEGER,
  nome VARCHAR(50),
  telefone VARCHAR(25),
  telefone_da_secretaria VARCHAR(25),
  login VARCHAR(50),
  senha VARCHAR(20),
  FOREIGN KEY (id_operadores) REFERENCES operadores(id) ON DELETE RESTRICT
);

CREATE TABLE servico_de_agendas (
```

```
id INTEGER PRIMARY KEY,  
id_operador INTEGER,  
nome VARCHAR(100),  
login VARCHAR(50),  
senha VARCHAR(50),  
FOREIGN KEY (id_operador) REFERENCES operadores(id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE convenios (  
id INTEGER PRIMARY KEY,  
id_medico INTEGER,  
nome VARCHAR(50),  
login VARCHAR(50),  
senha VARCHAR(50),  
FOREIGN KEY (id_medico) REFERENCES medicos(id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE clientes (  
id INTEGER PRIMARY KEY,  
nome VARCHAR(50) NOT NULL,  
telefone_whatsApp VARCHAR(25),  
e_mail VARCHAR(50),  
login VARCHAR(50) NOT NULL,  
senha VARCHAR(20) NOT NULL,  
id_convenio INTEGER,  
dateTime TIMESTAMP,  
matricula_no_convenio VARCHAR(50),  
FOREIGN KEY (id_convenio) REFERENCES convenios(id) ON DELETE RESTRICT,  
UNIQUE (login) -- índice único para login  
);
```

```
CREATE TABLE integracao (  
id INTEGER PRIMARY KEY,  
endereco_do_site VARCHAR(255) NOT NULL,  
login VARCHAR(50) NOT NULL,  
senha VARCHAR(20) NOT NULL,  
status BOOLEAN NOT NULL  
);
```

```
CREATE TABLE expediente_do_medico_data (  
id_medico INTEGER NOT NULL,  
data TIMESTAMP NOT NULL, -- data e hora com timestamp  
hora_inicial TIMESTAMP NOT NULL, -- hora inicial com timestamp  
hora_final TIMESTAMP NOT NULL, -- hora final com timestamp  
FOREIGN KEY (id_medico) REFERENCES medicos(id) ON DELETE RESTRICT,  
PRIMARY KEY (id_medico, data, hora_inicial) -- chave primária composta  
);
```

```
CREATE TABLE formas_de_pagamento (  
id INTEGER PRIMARY KEY,  
Nome VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE agenda (  
id INTEGER PRIMARY KEY,  
id_Medico INTEGER NOT NULL
```

```
id_medico INTEGER NOT NULL,
id_cliente INTEGER NOT NULL,
dateTime TIMESTAMP NOT NULL, -- data e hora com timestamp
dateTime_confirmacao TIMESTAMP NOT NULL, -- confirmação com timestamp
id_convenio INTEGER,
id_status_da_agenda_ou_consulta INTEGER NOT NULL,
id_formas_de_pagamento INTEGER,
dateTime_criacao TIMESTAMP NOT NULL, -- criação com timestamp
observacoes VARCHAR(255),
FOREIGN KEY (id_Medico) REFERENCES medicos(id) ON DELETE RESTRICT,
FOREIGN KEY (id_Cliente) REFERENCES clientes(id) ON DELETE RESTRICT,
FOREIGN KEY (id_convenio) REFERENCES convenios(id) ON DELETE RESTRICT,
FOREIGN KEY (id_status_da_agenda_ou_consulta) REFERENCES
status_da_agenda_ou_consulta(id) ON DELETE RESTRICT,
FOREIGN KEY (id_formas_de_pagamento) REFERENCES formas_de_pagamento(id)
ON DELETE RESTRICT
);

CREATE TABLE consulta (
id INTEGER PRIMARY KEY,
id_agenda INTEGER,
id_cliente INTEGER NOT NULL,
id_medico INTEGER NOT NULL,
dateTime TIMESTAMP NOT NULL, -- data e hora com timestamp
id_convenio INTEGER,
id_forma_de_pagamento INTEGER,
id_status_da_agenda_ou_consulta INTEGER NOT NULL,
Observacao VARCHAR(255),
FOREIGN KEY (id_agenda) REFERENCES agenda(id) ON DELETE RESTRICT,
FOREIGN KEY (id_cliente) REFERENCES clientes(id) ON DELETE RESTRICT,
FOREIGN KEY (id_medico) REFERENCES medicos(id) ON DELETE RESTRICT,
FOREIGN KEY (id_convenio) REFERENCES convenios(id) ON DELETE RESTRICT,
FOREIGN KEY (id_forma_de_pagamento) REFERENCES formas_de_pagamento(id) ON
DELETE RESTRICT
);

-- Observações:
-- * Adapte o script de acordo com o seu dialeto PostgreSQL.
-- * As colunas "dateTime_disponivel" e "Observacao" na tabela "clientes"
possuem
-- caracteres especiais. Verifique se a sua configuração de codificação
de caracteres
-- está correta para evitar erros ao inserir dados.

-- A tabela __dm_xtable__ é usada para testes do modelo de Tb_Table.pas

CREATE TABLE __dm_xtable__ (
id SERIAL PRIMARY KEY,
nome VARCHAR(50) NOT NULL,
endereco VARCHAR(50),
cnpj VARCHAR(18),
cpf VARCHAR(14).
```

```
cep VARCHAR(10),  
valor_SMALLINT SMALLINT,  
valor_Integer Integer,  
valor_FLOAT8 DOUBLE PRECISION,  
Data_1 TIMESTAMP,  
hora_1 TIMESTAMP,  
hora_2 TIMESTAMP  
);
```

Script SQL PostgreSQL para criar os relacionamentos

```
-- Adicionando chaves estrangeiras

-- Tabela medicos
ALTER TABLE medicos ADD CONSTRAINT fk_medicos_operadores FOREIGN KEY
(id_operadores) REFERENCES operadores(id);

-- Tabela servico_de_agendas
ALTER TABLE servico_de_agendas ADD CONSTRAINT
fk_servico_de_agendas_operadores FOREIGN KEY (id_operador) REFERENCES
operadores(id);

-- Tabela convenios
ALTER TABLE convenios ADD CONSTRAINT fk_convenios_medicos FOREIGN KEY
(id_medico) REFERENCES medicos(id);

-- Tabela clientes
ALTER TABLE clientes ADD CONSTRAINT fk_clientes_convenios FOREIGN KEY
(id_convenio) REFERENCES convenios(id);

-- Tabela agenda
ALTER TABLE agenda ADD CONSTRAINT fk_agenda_medicos FOREIGN KEY
(id_Medico) REFERENCES medicos(id);
ALTER TABLE agenda ADD CONSTRAINT fk_agenda_clientes FOREIGN KEY
(id_Cliente) REFERENCES clientes(id);
ALTER TABLE agenda ADD CONSTRAINT fk_agenda_convenios FOREIGN KEY
(id_convenio) REFERENCES convenios(id);
ALTER TABLE agenda ADD CONSTRAINT
fk_agenda_status_da_agenda_ou_consulta FOREIGN KEY
(id_status_da_agenda_ou_consulta) REFERENCES
status_da_agenda_ou_consulta(id);
ALTER TABLE agenda ADD CONSTRAINT fk_agenda_formas_de_pagamento FOREIGN
KEY (id_formas_de_pagamento) REFERENCES formas_de_pagamento(id);

-- Tabela consulta
ALTER TABLE consulta ADD CONSTRAINT fk_consulta_agenda FOREIGN KEY
(id_agenda) REFERENCES agenda(id);
ALTER TABLE consulta ADD CONSTRAINT fk_consulta_clientes FOREIGN KEY
(id_cliente) REFERENCES clientes(id);
ALTER TABLE consulta ADD CONSTRAINT fk_consulta_medicos FOREIGN KEY
(id_medico) REFERENCES medicos(id);
ALTER TABLE consulta ADD CONSTRAINT fk_consulta_convenios FOREIGN KEY
(id_convenio) REFERENCES convenios(id);
ALTER TABLE consulta ADD CONSTRAINT fk_consulta_forma_de_pagamento
FOREIGN KEY (id_forma_de_pagamento) REFERENCES formas_de_pagamento(id);
ALTER TABLE consulta ADD CONSTRAINT
fk_consulta_status_da_agenda_ou_consulta FOREIGN KEY
(id_status_da_agenda_ou_consulta) REFERENCES
status_da_agenda_ou_consulta(id);
```

- **Observações:**

- Essas chaves estrangeiras garantem a integridade referencial das tabelas, o que significa que os dados nas tabelas relacionadas serão consistentes.
- Por exemplo, a chave estrangeira `fk_medico_operador` na tabela `medico` garante que o valor da coluna `id_operador` em uma linha da tabela `medico` sempre existirá em uma linha da tabela `operador`.

Script SQL PostgreSQL para criar os índices das tabelas

```
-- Índices para a tabela operadores
CREATE INDEX idx_operadores_nome ON operadores (nome);

-- Índices para a tabela medicos
CREATE INDEX idx_medicos_nome ON medicos (nome);
CREATE INDEX idx_medicos_id_operadores ON medicos (id_operadores);

-- Índices para a tabela servico_de_agendas
CREATE INDEX idx_servico_de_agendas_nome ON servico_de_agendas (nome);
CREATE INDEX idx_servico_de_agendas_id_operador ON servico_de_agendas
(id_operador);

-- Índices para a tabela convenios
CREATE INDEX idx_convenios_nome ON convenios (nome);
CREATE INDEX idx_convenios_id_medico ON convenios (id_medico);

-- Índices para a tabela clientes
CREATE INDEX idx_clientes_nome ON clientes (nome);
CREATE INDEX idx_clientes_login ON clientes (login);
CREATE INDEX idx_clientes_id_convenio ON clientes (id_convenio);

-- Índice para a coluna id_medico na tabela expediente_do_medico_data
CREATE INDEX idx_expediente_do_medico_data_id_medico ON
expediente_do_medico_data (id_medico);

-- Índice para a coluna dateTime na tabela expediente_do_medico_data
CREATE INDEX idx_expediente_do_medico_data_dataTime ON
expediente_do_medico_data (dateTime);

-- Índice para a coluna id_expediente_do_medico_data na tabela
expediente_do_medico_horas
CREATE INDEX
idx_expediente_do_medico_horas_id_expediente_do_medico_data ON
expediente_do_medico_horas (id_expediente_do_medico_data);

-- Índices para a tabela formas_de_pagamento
CREATE INDEX idx_formas_de_pagamento_nome ON formas_de_pagamento
(Nome);

-- Índices para a tabela agenda
CREATE INDEX idx_agenda_id_Medico ON agenda (id_Medico);
CREATE INDEX idx_agenda_id_Cliente ON agenda (id_Cliente);
CREATE INDEX idx_agenda_dataTime ON agenda (dateTime);
CREATE INDEX idx_agenda_id_convenio ON agenda (id_convenio);
```

```
CREATE INDEX idx_agenda_id_status_da_agenda_ou_consulta ON agenda
(id_status_da_agenda_ou_consulta);
CREATE INDEX idx_agenda_id_formas_de_pagamento ON agenda
(id_formas_de_pagamento);

-- Índices para a tabela consulta
CREATE INDEX idx_consulta_id_agenda ON consulta (id_agenda);
CREATE INDEX idx_consulta_id_cliente ON consulta (id_cliente);
CREATE INDEX idx_consulta_id_medico ON consulta (id_medico);
CREATE INDEX idx_consulta_dataTime ON consulta (dateTime);
CREATE INDEX idx_consulta_id_convenio ON consulta (id_convenio);
CREATE INDEX idx_consulta_id_forma_de_pagamento ON consulta
(id_forma_de_pagamento);
CREATE INDEX idx_consulta_id_status_da_agenda_ou_consulta ON consulta
(id_status_da_agenda_ou_consulta);
```

- **Observações:**

- Após criar as tabelas e as chaves estrangeiras, você poderá inserir dados nas tabelas. Os dados que você insere devem corresponder aos atributos das classes no diagrama de classes.
- Por exemplo, para inserir um médico na tabela medico, você precisará fornecer os valores para os seguintes atributos:
 - id (auto-incrementado)
 - id_operador
 - nome
 - telefone
 - telefone_secretaria
 - login
 - senha
- Exemplos de consultas que você pode usar para trabalhar com os dados no banco de dados:
 - Para obter uma lista de todos os médicos:

```
SELECT *
FROM medico;
```

- Para obter uma lista de todos os médicos de um determinado hospital:

```
SELECT *
FROM medico
WHERE id_hospital = 1;
```

- Para obter uma lista de todas as consultas de um determinado paciente:

```
SELECT *  
FROM consulta  
WHERE id_cliente = 1;
```

- Para obter uma lista de todas as consultas de um determinado médico:

```
SELECT *  
FROM consulta  
WHERE id_medico = 1;
```

Compartilhar o acesso ao banco de dados

- Conceder acesso ao banco de dados:

```
GRANT ALL PRIVILEGES ON DATABASE mydatabase TO mypartner;
```

- Este comando concederá ao seu parceiro todos os privilégios no banco de dados mydatabase.

Ferramenta que pode ser usada para executar os scripts acima?

- *psql*
 - Para executar os scripts acima usando o *psql*, você precisará conectar-se ao banco de dados PostgreSQL. Você pode fazer isso usando o comando *psql*:

```
psql -h <host> -p <port> -d <database> -U <username>
```

- Por exemplo, para conectar-se ao banco de dados PostgreSQL chamado *mydatabase* no host *localhost* na porta *5432*, você usaria o seguinte comando:

```
psql -h localhost -p 5432 -d mydatabase -U myuser
```

- Uma vez conectado ao banco de dados, você pode executar os scripts acima usando o comando \i:

```
\i create_tables.sql
```

- Nota:
 - Este comando irá executar o script *create_tables.sql*, que criará as tabelas necessárias para o diagrama de classes.
- *dbeaver-ce*
 - O *DBeaver Community Edition* (DBeaver CE) é uma ferramenta de gerenciamento de banco de dados relacional de código aberto. Ele oferece suporte a vários bancos de dados, como MySQL, PostgreSQL, SQLite, Oracle, Microsoft SQL Server, e muitos outros. O DBeaver CE é uma versão gratuita e de código aberto da ferramenta, projetada para ser usada por desenvolvedores, administradores de banco de dados e qualquer pessoa envolvida no gerenciamento e manipulação de bancos de dados.
 - Principais características do DBeaver CE:
 - Conectividade com Múltiplos Bancos de Dados: Oferece suporte a uma ampla variedade de sistemas de gerenciamento de banco de dados, permitindo que os usuários se conectem a diferentes bancos de dados a partir de uma única interface.
 - Interface Gráfica Intuitiva: Possui uma interface de usuário gráfica intuitiva que facilita o desenvolvimento, gerenciamento e consulta de bancos de dados.
 - Editor SQL: Inclui um editor SQL poderoso com realce de sintaxe, conclusão automática e outras funcionalidades que facilitam a escrita e execução de consultas SQL.
 - Gerenciamento de Metadados: Permite a visualização e edição de metadados do banco de dados, como tabelas, índices, procedimentos armazenados, entre outros.
 - Exportação e Importação de Dados: Facilita a transferência de dados entre diferentes fontes, com suporte a várias opções de exportação e importação.
 - Visualização de Dados: Oferece recursos visuais para a visualização de dados, incluindo gráficos e ferramentas de exploração de dados.
 - Suporte a Plugins: Permite a extensão de funcionalidades por meio de plugins, proporcionando flexibilidade para atender a diferentes necessidades.

- O DBeaver CE é uma escolha popular devido à sua versatilidade e ao fato de ser gratuito e de código aberto. No entanto, é importante observar que existem versões comerciais do DBeaver (DBeaver EE) que oferecem recursos adicionais e suporte aprimorado, mas a versão Community Edition é uma opção robusta para muitos desenvolvedores e administradores de banco de dados.

Referências

1. [Gráfico criado com projeto mermaid](#)
2. [PostgreSQL: The World's Most Advanced Open Source](#)

[